

Name: Momina Idrees

CINC NO: 35202 - 2172367-8

Machine Learning – CARS Dataset (Classification)

DOCUMENTATION

The following tasks classify cars by their body type using a real-world automotive dataset containing both numerical and categorical features. Multiple machine learning models are applied after thorough data preprocessing, feature engineering, and exploratory data analysis to evaluate and compare their performance.

This document doesn't only include the description of each step but results are also attached.

Libraries used:

```
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt
from sklearn.compose import ColumnTransformer
from sklearn.preprocessing import OneHotEncoder
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.ensemble import RandomForestClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.svm import LinearSVC
from sklearn.metrics import f1_score, accuracy_score
from sklearn.metrics import classification_report, confusion_matrix
```

TASKS:

Loading Dataset:

The data is loaded as a data frame using **pandas** packages. Since the data is too large so we are taking random sample data of dataset to train the model.

Data Study & Analyzing:

Initial data analysis is performed to understand the structure and quality of the dataset. This includes checking the dataset **shape**, **column names**, **data types**, **missing values**, **duplicate records**, and basic **statistical summaries** of numerical features.

The dataset includes **multiple data types**, including numerical, categorical, and Boolean variables, The target variable **Body type** is identified for classification, while the remaining columns are treated as input features (X) for model training.

Results: Shape of data set: (50000, 32)

INFO OF DATASET:

#	Column	Non-Null Count	Dtype
0	Brand	50000 non-null	object
1	Model	50000 non-null	object
2	BodyType	50000 non-null	object
3	Category	50000 non-null	object
4	Drive	49725 non-null	object
5	Doors	50000 non-null	int64
6	Km	50000 non-null	int64
7	YearBuilt	50000 non-null	int64
8	Power_kW_HP	50000 non-null	int64
9	EngineCapacity	50000 non-null	float64
10	Cylinders	49924 non-null	float64
11	EmptyWeight	49984 non-null	float64
12	EmissionClass	49932 non-null	object
13	FuelType	49981 non-null	object
14	Price	50000 non-null	int64
15	Seats	50000 non-null	float64
16	FullServiceHistory	50000 non-null	bool
17	Transmission	50000 non-null	object
18	Gears	35716 non-null	float64
19	CO2Emission	49316 non-null	float64
20	Color	50000 non-null	object
21	Warranty	50000 non-null	object
22	APK	31895 non-null	object
23	FuelConsumption	7655 non-null	float64
24	PreviousOwners	15886 non-null	float64
25	NonSmokingCar	50000 non-null	bool
26	APK_month_diff	50000 non-null	int64
27	EngineCapacity_Corrected	50000 non-null	float64
28	A1	50000 non-null	float64
29	A2	50000 non-null	float64
30	A3	50000 non-null	float64
31	A4	50000 non-null	float64

dtypes: bool(2), float64(13), int64(6), object(11)

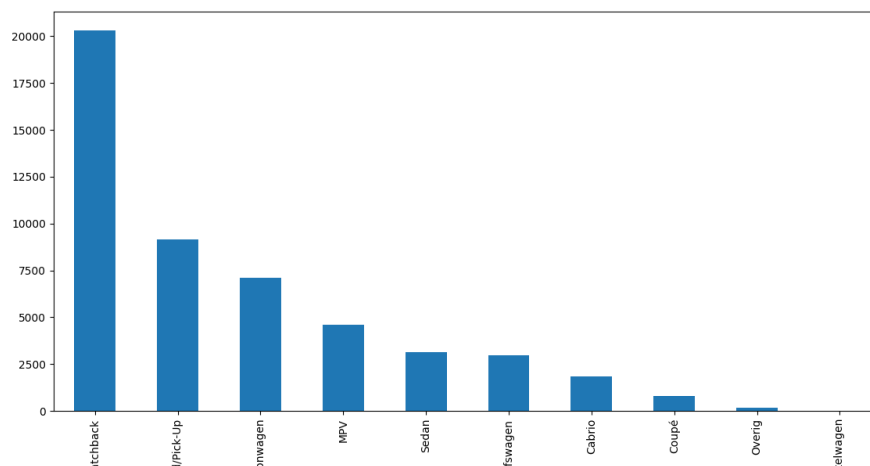
MISSING & DUPLICATES

Finding null values : Brand
Model 0
BodyType 0
Category 0
Drive 275
Doors 0
Km 0
YearBuilt 0
Power_kW_HP 0
EngineCapacity 0
Cylinders 76
EmptyWeight 96
EmissionClass 68
FuelType 19
Price 0
Seats 0
FullServiceHistory 0
Transmission 0
Gears 14284
CO2Emission 684
Color 0
Warranty 0
APK 18105
FuelConsumption 42945
PreviousOwners 34914
NonSmokingCar 0
APK_month_diff 0
EngineCapacity_Corrected 0
A1 0
A2 0
A3 0
A4 0
dtype: int64

Finding Duplicates of dataset : 305

Visualization through charts:

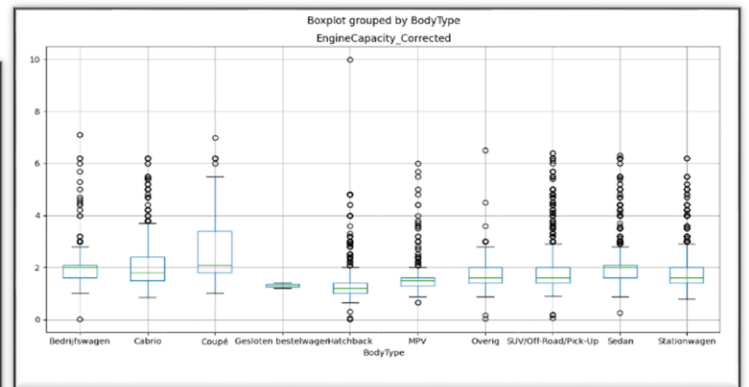
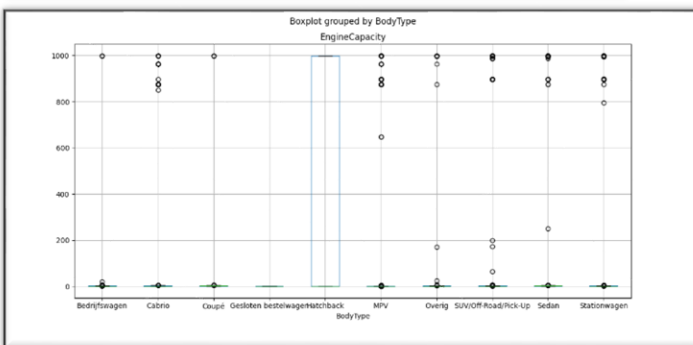
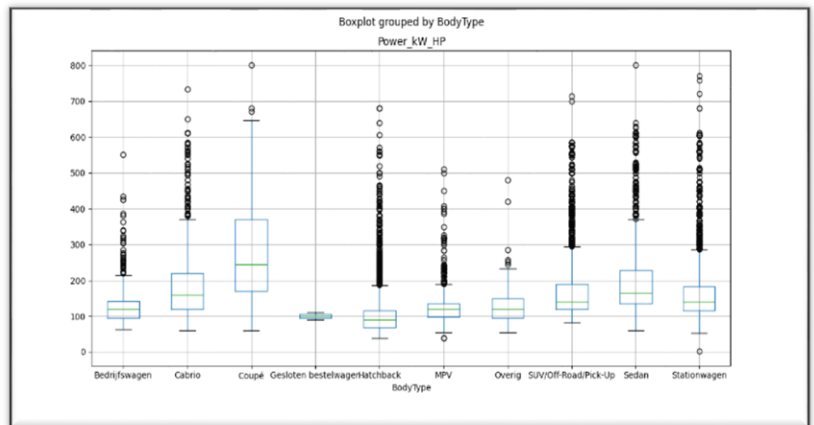
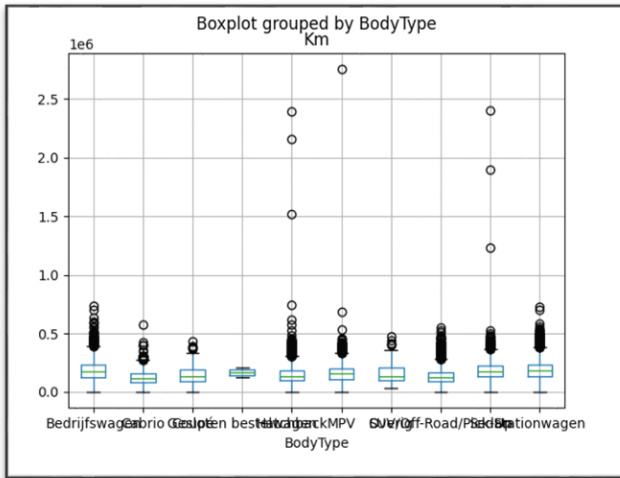
CATAGORIES INCLUDED IN TARGET COLUMN



This bar chart is visualizing the distribution of car body types in the dataset, which highlights a clear **class imbalance**, where **Hatchback** and **SUV** categories dominate the dataset, while body types such as **Coupe** and **Gesloten bestelwagen** are significantly less dominant.

Uneven class distributions can bias machine learning models toward majority classes (like **Hatchback**) and negatively affect prediction performance for minority classes (like **Overig**).

NUMERICAL COLUMNS AGAINST TARGET COLUMN



These box plots show how important numerical features **vary across different car body types**. From the plots, it is clear that features like engine capacity, power, doors, and seats **change depending on the body type**, which means they are **useful for predicting car categories**

The visualizations also highlight the **outliers**, in form of dots, especially in the mileage (Km) feature, where some cars have extremely high values. Such outliers can negatively affect model training by introducing bias and instability. This analysis helps identify important features, understand class-wise differences, and justify the need for preprocessing steps such as outlier handling and feature scaling before model training.

Data Cleaning:

This block of code performs essential data cleaning to improve data quality before model training.

First, **duplicate rows** are removed to prevent the model from learning repeated information, which can bias results.

Secondly, **unnecessary columns** such as APK, Category, and APK_month_diff are **dropped** because they either contain very little data or do not help in predicting the car body type.

Thirdly, the separate lists are made of numerical and categorical data in order to deal with the column in right way.

- Num_col [] for numerical columns
- Cat_col [] for categorical column.

Missing values in numeric columns are filled using the **median** to avoid distortion from extreme values, while missing values in **categorical columns** are replaced with the label "**unknown**" to keep the data consistent. As a result, the dataset contains no missing values, allowing machine learning models to train on complete, clean, and reliable data, which leads to more stable and accurate predictions.

Results:

```
Shape before removing duplicate rows : (50000, 32)
Shape After removing duplicate rows : (49695, 32)
Before removing the unnecessary columns , total columns were (49695, 32)
After removing the unnecessary columns , total columns were (49695, 29)
```

Preprocessing:

The target column **BodyType** is removed from the input features to prepare the data for model training.

Machine learning models cannot understand text data, so **categorical features are converted into numerical form using One-Hot Encoding**.

One-Hot Encoding represents each category as a **combination of 0s and 1s** without repeating the order of categories

Numerical features such as mileage and engine capacity have very **large value ranges**, which can bias model learning. **Standard Scaling** is applied to bring all numerical features to a similar scale

Then a **ColumnTransformer** is applied to process numerical and categorical features inside a same method.

```
df_car_preprocess = ColumnTransformer(
    transformers=[
        ('n', StandardScaler(), num_col),
        ('c', OneHotEncoder(handle_unknown='ignore', sparse_output=True), cat_col)])
```

Within the **ColumnTransformer**,

A transformer **'n'** scales numerical columns using **StandardScaler**, converting all the numerical columns with wide range into a smaller and similar scale, ensuring that features like mileage, power, and engine capacity contribute evenly instead of overpowering the model.

A transformer **'c'** applies **OneHotEncoder** to categorical columns, converting them into machine-readable form while preserving category distinctions.

This combined transformation helps the model learn more reliable and unbiased patterns from the data.

This preprocessing ensures **clean, consistent, and model-ready data**, leading to better learning and more accurate predictions.

Feature Engineering:

Feature engineering is used to create new meaningful **features from existing data** to improve model understanding.

For example, in this project:

A new feature **Vehicle_Year** is created by subtracting YearBuilt from the current year (2018 as the data is from 2006 to 2018). This converts the build year into vehicle age, which is more informative because models understand age better than raw year values. Older and newer cars of the same body type can differ in size, design, and performance, and this feature helps capture that difference.

Modeling:

First, the dataset is **divided** into:

- $X \rightarrow$ input features (all columns except BodyType)
- $Y \rightarrow$ target column (BodyType)

The data is then split into training and testing sets using **train_test_split**:

- **80%** of the data is used for **training**.
- **20%** of the data is reserved for **testing**.

This method divides the data into **two parts**.

The first part contains the **training data, which is 80% of both X and Y**. The models are trained using this data.

The second part contains the testing data, which is **20% of both X and Y, and it is used to evaluate the model's performance** during prediction.

Here is the shape of training and testing data

```
shape of x_train : (39756, 30)
shape of y_train : (39756,)
shape of x_test : (9939, 30)
shape of y_test : (9939,)
```

After splitting the data, preprocessing is defined again because models must only learn transformations from the training data.

Models Trained:

We train four models, and each model is placed inside a pipeline along with the data preparation steps.

1. Logistic Regression

```
LR_pipeline = Pipeline(
    steps=[
        ('preprocessing', df_car_preprocess),
        ('model', LogisticRegression(class_weight='balanced', max_iter=1000))]
)
LR_pipeline.fit(x_train, y_train)
```

- A basic linear model used as a starting point
- Class weights are adjusted to handle imbalanced data
- More iterations are allowed so the model can fully learn

2. Random Forest

```
RF_pipeline = Pipeline(
    steps=[
        ('preprocessing', df_car_preprocess),
        ('model', RandomForestClassifier())
    ]
)
RF_pipeline.fit(x_train, y_train)
```

- Combines many decision trees to make predictions
- Can capture complex patterns in the data

3. Decision Tree

```
DT_pipeline = Pipeline(
    steps=[
        ('preprocessing', df_car_preprocess),
        ('model', DecisionTreeClassifier())
    ]
)
DT_pipeline.fit(x_train, y_train)
```

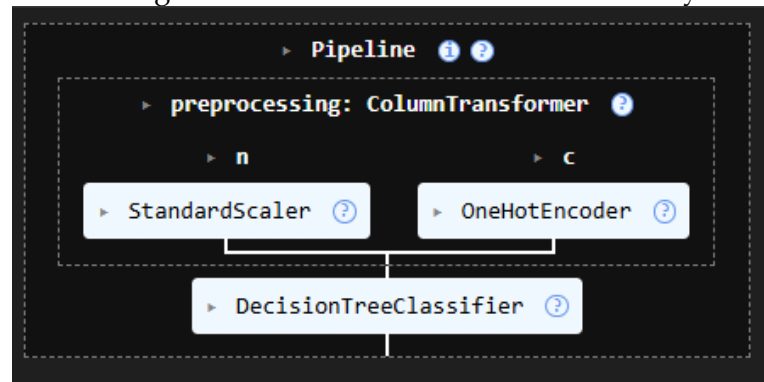
- Learns simple rules from the data to make predictions
- Easy to understand, but can sometimes fit the training data too closely

4. Linear SVC

```
L_SVC_pipeline = Pipeline(
    steps=[
        ('preprocessing', df_car_preprocess),
        ('model', LinearSVC(class_weight='balanced'))
    ]
)
L_SVC_pipeline.fit(x_train, y_train)
```

Linear SVC is used because it separates classes using a straight line and works well with high-dimensional data.

A **pipeline-based** approach is used to train the models. Each pipeline ensures that preprocessing is applied before model training and the transformations are consistently used during prediction



Evaluation

Now that the models are trained Here we test all the models trained above. For every model:

- ✓ We predict results on test data.
- ✓ We check the classification report.
- ✓ We display the confusion matrix.
- ✓ We check accuracy score

1. Logistic Regression:

Classification report:

The classification report of `y_test` and logistic regression prediction:

	precision	recall	f1-score	support
Bedrijfswagen	0.97	0.90	0.94	582
Cabrio	0.82	0.88	0.85	404
Coupé	0.56	0.87	0.68	164
Gesloten bestelwagen	0.00	0.00	0.00	0
Hatchback	0.95	0.79	0.87	4105
MPV	0.83	0.89	0.86	853
Overig	0.04	0.33	0.07	33
SUV/Off-Road/Pick-Up	0.95	0.96	0.96	1785
Sedan	0.69	0.80	0.74	639
Stationwagen	0.70	0.79	0.74	1374
accuracy			0.84	9939
macro avg	0.65	0.72	0.67	9939
weighted avg	0.88	0.84	0.85	9939

The insights from this classification report are that this model predicts large classes very well, but struggles with smaller classes.

here are the results with their f1 score

- ✓ Very good predictions on:
- ✓ SUV / Off-Road / Pick-Up → 96%
- ✓ Bedrijfswagen → 94%
- ✓ Hatchback → 87%
- ✓ MPV → 86%

The model is strong where there is plenty of data.

- ✓ Average performance on:
- ✓ Sedan → 74%
- ✓ Stationwagen → 74%

Since both have similar shape, size and features. The model gets confused by similar but slightly different body types:

- ✓ Weak prediction on:
- ✓ Coupé → 68%
- ✓ Overig → 07%
- ✓ Gesloten bestelwagen → 0%

These classes barely had any data. The model is not reliable for rare classes.

Accuracy:

Accuracy = 84%

Which means it is giving mostly right predictions. specially on larger data .

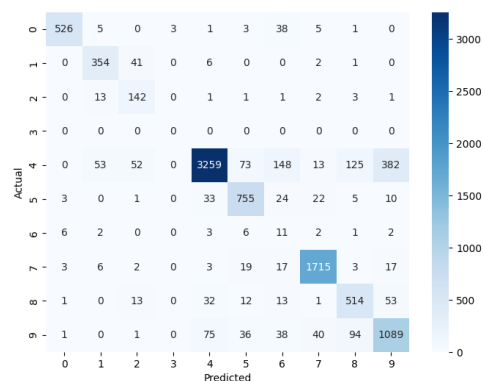
Macro F1 = 67%

which means the model performs only moderately when all classes are given equal importance.

Weighted F1 = 85%

which means that the model works well for the most common classes, which have more samples.

Confusion matrix:



The diagonal values show correct predictions, and most values are on the diagonal, meaning the model predicts many samples correctly.

Large diagonal numbers indicate strong performance for common classes like Hatchback, SUV, and Stationwagen.

Off-diagonal values show where the model gets confused, mostly between similar body types.

2. Random Forest

	precision	recall	f1-score	support
Bedrijfswagen	0.97	0.98	0.97	582
Cabrio	0.88	0.91	0.89	404
Coupé	0.83	0.59	0.69	164
Hatchback	0.91	0.96	0.94	4105
MPV	0.91	0.84	0.87	853
Overig	1.00	0.15	0.26	33
SUV/Off-Road/Pick-Up	0.92	0.98	0.95	1785
Sedan	0.89	0.75	0.82	639
Stationwagen	0.84	0.78	0.81	1374
accuracy			0.91	9939
macro avg	0.91	0.77	0.80	9939
weighted avg	0.90	0.91	0.90	9939

The insights from this classification report show that this model performs very well on large and common classes, while performance drops for smaller and rare classes.

❖ Very good predictions on:

- ✓ Bedrijfswagen → 97%
- ✓ SUV / Off-Road / Pick-Up → 95%
- ✓ Hatchback → 94%
- ✓ MPV → 87%
- ✓ Cabrio → 89%

The model is very strong where there is plenty of data, showing high precision and recall for majority classes.

❖ Average performance on:

- ✓ Sedan → 82%
- ✓ Stationwagen → 81%

These classes have similar shape, size, and features, which causes confusion between similar but slightly different body types.

❖ Weak predictions on:

- ✓ Coupe → 69%
- ✓ Overig → 26%

These classes have very limited data, so the model struggles to learn reliable patterns and is not dependable for rare classes.

Accuracy:

Accuracy = 91%

The model gives mostly correct predictions, especially for large and well-represented classes.

Macro F1 = 80%

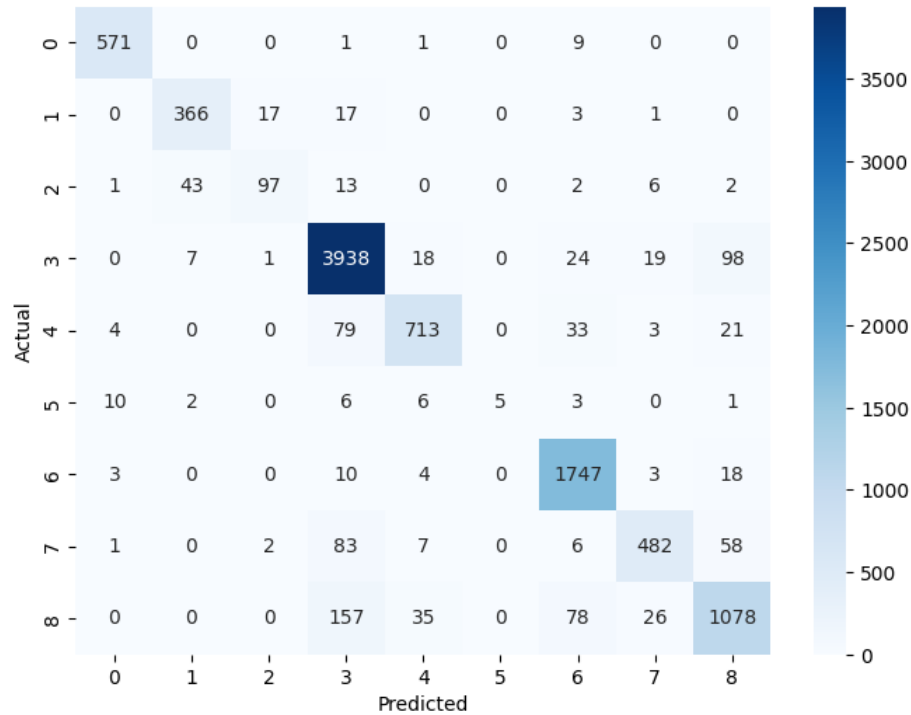
The model performs reasonably when all classes are given equal importance, but weaker classes reduce the overall score.

Weighted F1 = 90%

The model performs very well on the most common classes, which have more samples and higher

influence.

Confusion Metrix:



3. Decision Tree

Classification report:

	precision	recall	f1-score	support
Bedrijfswagen	0.95	0.95	0.95	582
Cabrio	0.88	0.89	0.89	404
Coupé	0.74	0.74	0.74	164
Hatchback	0.93	0.92	0.92	4105
MPV	0.80	0.81	0.81	853
Overig	0.28	0.21	0.24	33
SUV/Off-Road/Pick-Up	0.93	0.93	0.93	1785
Sedan	0.76	0.76	0.76	639
Stationwagen	0.78	0.78	0.78	1374
accuracy			0.88	9939
macro avg	0.78	0.78	0.78	9939
weighted avg	0.88	0.88	0.88	9939

❖ Strongly Predicted

- ✓ Bedrijfswagen → 95%
- ✓ SUV / Off-Road / Pick-Up → 93%
- ✓ Hatchback → 92%
- ✓ Cabrio → 89%

These classes have high precision and recall, meaning predictions are both accurate and complete.

❖ Average Performance

- ✓ Sedan → 76%
- ✓ Stationwagen → 78%
- ✓ Coupé → 74%
- ✓ MPV → 81%

These body types are similar in shape and features, which causes some confusion between them.

❖ Weakly Predicted Class

- ✓ Overig → 24%

This class has very few samples, so the model cannot learn and predict .

Accuracy :

Accuracy = 88%

The model predicts most cars correctly.

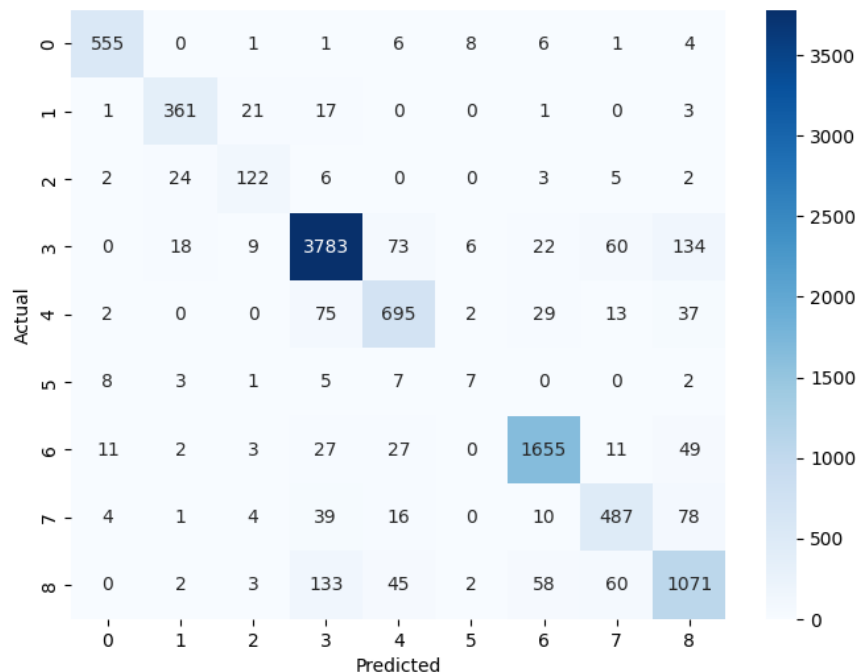
Weighted F1 = 88%

The model performs very well on common classes with more data.

Macro F1 = 78%

When all classes are treated equally, performance drops, showing weaker results on small classes.

Confusion Metrix:



4. Linear SVC:

Classification report:

	precision	recall	f1-score	support
Bedrijfswagen	0.98	0.98	0.98	582
Cabrio	0.84	0.83	0.83	404
Coupé	0.58	0.76	0.66	164
Hatchback	0.92	0.88	0.90	4105
MPV	0.86	0.86	0.86	853
Overig	0.08	0.27	0.12	33
SUV/Off-Road/Pick-Up	0.95	0.98	0.96	1785
Sedan	0.67	0.68	0.68	639
Stationwagen	0.71	0.71	0.71	1374
accuracy			0.86	9939
macro avg	0.73	0.77	0.74	9939
weighted avg	0.87	0.86	0.86	9939

Accuracy:

Accuracy = 86%

→ The model predicts most vehicles correctly.

Weighted F1 = 86%

→ The model works well on common classes with more data.

Macro F1 = 74%

→ When all classes are treated equally, performance is lower, indicating weaker predictions for minority classes.

❖ Strongly Predicted

- ✓ Bedrijfswagen → 95%
- ✓ SUV / Off-Road / Pick-Up → 93%
- ✓ Hatchback → 92%
- ✓ Cabrio → 89%

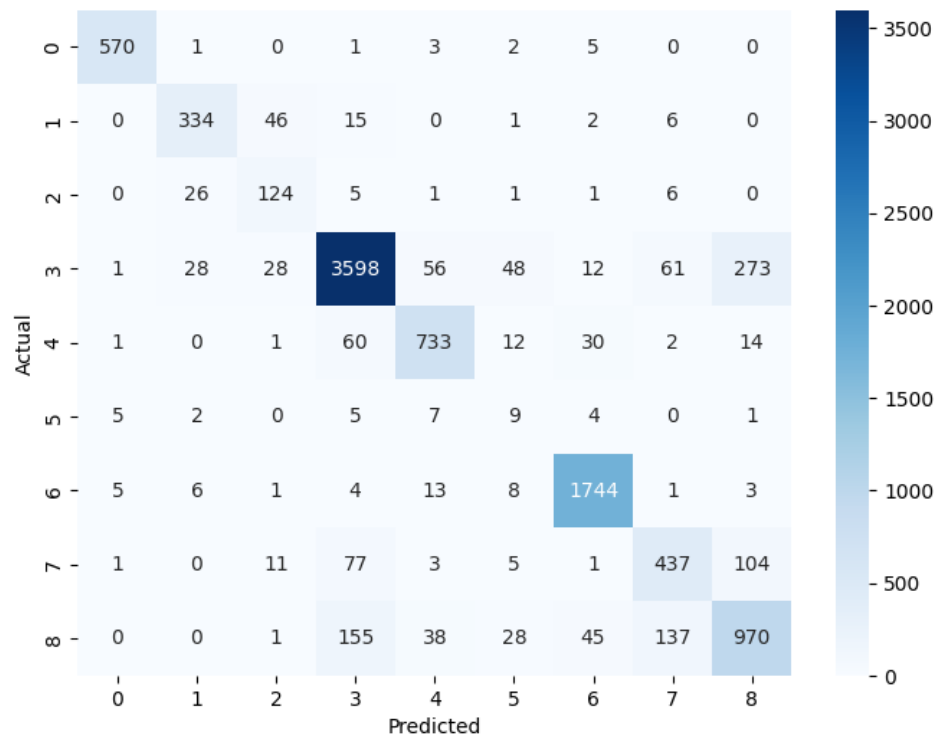
❖ Average Performance

- ✓ Sedan → 76%
- ✓ Stationwagen → 78%
- ✓ Coupé → 74%
- ✓ MPV → 81%

❖ Weakly Predicted Class

- ✓ Overig → 24%

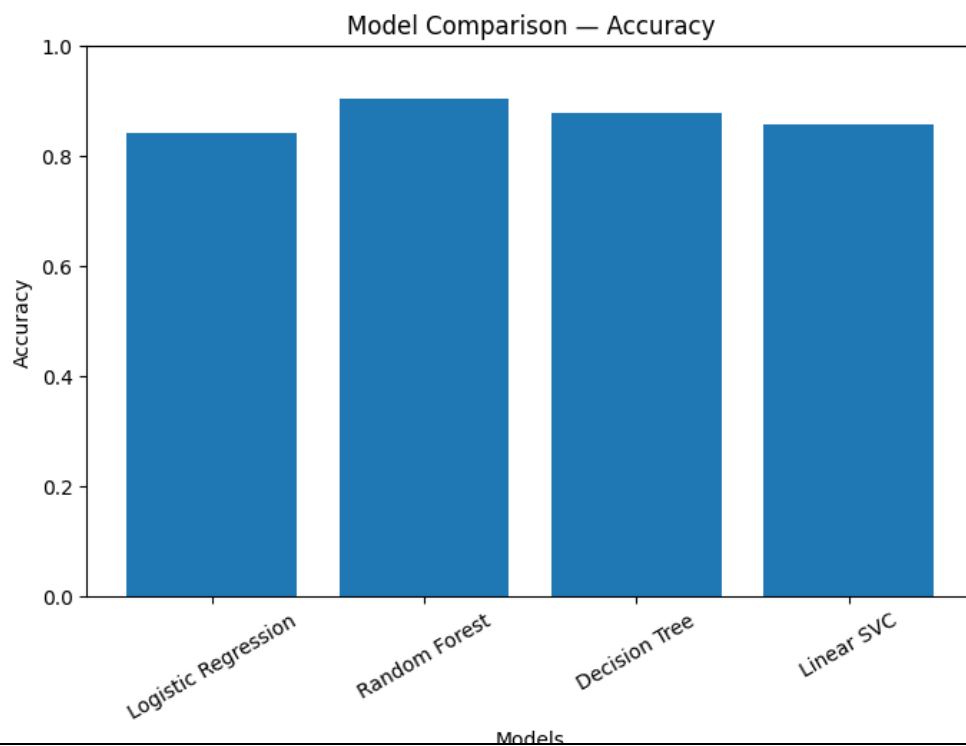
Confusion Metrix:



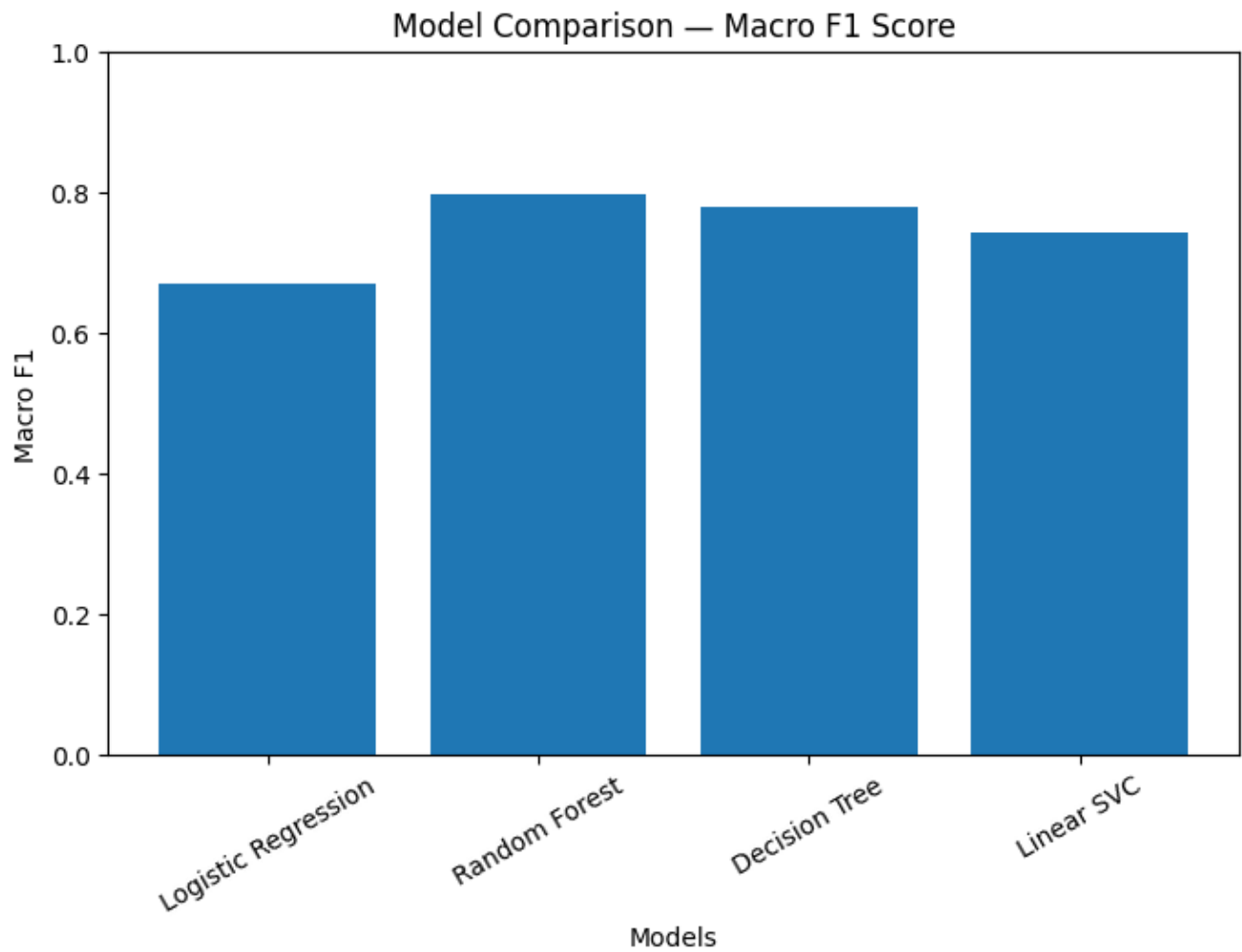
Overall, all models perform reasonably well, with accuracy ranging from **84% to 91%**, and the highest accuracy achieved by the **Random forest**.

Comparison of all models accuracy:

Accuracy:



Macro f-1 score:



Weight f1 score:

